

- Note Synapse Documentation
 - 🖋️ Editor & Writing
 - 📁 Organization
 - 🧠 AI & Thinking
 - ⚡ Productivity
 - 🧑💻 Developer
 - 🖍️ Examples
- Hands-On: Tag Algebra (Virtual Folders)
 - The Concept: Filters Create Hierarchy
 - Problem
 - Walkthrough: Creating a “Virtual” Hierarchy
 - Tag Filters: Your Saved Views
 - 1. The Filter Strip
 - 2. Hierarchy Mode (Auto-Organization)
 - 3. Pinning for Speed
 - Automation: Associated Tag Prompts
 - Use Case: Research Papers
- Hands-On: Web Clipper & API Vibe Coding
 - The “Readability” Toggle
 - Problem
 - Walkthrough
 - Dealing with Images
 - Method 1: The “Rich” Clip
 - Method 2: Fetching Later
 - Hands-On: Clipping API Docs for Vibe Coding
 - Usage
- Hands-On: Calendar & Time Blocking
 - Drag & Drop Rescheduling
 - Problem
 - Walkthrough
 - The Timeline View (GTD)
- Tree Conversations: Git for Thoughts

- The “Forking” Workflow
 - Problem
 - Solution: Fork It
 - Navigating the Tree
 - The Graph View
 - Multi-Select Actions
 - Use Case: Text Adventure Game
 - Context Pruning
- Hands-On: Immersive Reading (PDFs)
 - The “Circle to Ask” Gesture
 - Problem
 - Entering Immersive Mode
 - Working with PDFs
 - 1. Opening a PDF
 - 2. The Side Handle (AI Panel)
 - The “Circle to Ask” Gesture
 - Problem
 - Walkthrough
 - The Scratchpad
 - Problem
 - Solution
 - Hands-On: Limiting Context Scope
 - Problem
 - Walkthrough
- Connecting to External Tools (MCP)
 - What is MCP?
 - Supported Transports
 - How to Connect: The Definitive Guide
 - 1. The “Magic” Way (Auto-Discovery)
 - 2. The “Simple” Way (Token Auth)
 - 3. The “Manual” Way (Advanced OAuth)
 - Debugging Tools (AI Introspection)
- Smart Model Matching (Multi-Capability Selection)
 - Problem
 - Solution: Capability-Based Priority

- How to Configure
- Matching Logic
- Supported Capabilities
- Hands-On: Agentic Research
 - The “Research Agent” Workflow
 - Problem
 - Walkthrough
 - Reviewing the Plan
- Hands-On: Block-Based Editing
 - The “Hidden” Drag Gesture
 - Problem
 - Solution
 - Walkthrough
 - Reordering Blocks
- Architecture
 - Technical Stack
 - Network Resilience (HTTP/3)
 - Agent Architecture
 - Local-First Data Model
- User Apps & “Vibe Coding”
 - The “Vibe Coding” Workflow
 - Tutorial: “Build a Weather Dashboard”
 - `window.Synapse` API Reference
 - 1. Database Access (SQL)
 - 2. State Management
 - 3. AI Integration (Multi-Part)
 - 3. Network Proxy
 - 4. Note Manipulation
 - Security Model

Note Synapse Documentation

The **Hands-On** Manual for Power Users.

Editor & Writing

- **Block-Based Editing**: The “hidden” drag gesture, reordering, and block types.

Organization

- **Tag Algebra**: Creating virtual folder hierarchies (`Tag A` + `Tag B`).

AI & Thinking

- **Tree Conversations**: “Git for Chat”, forking timelines, and pruning context.
- **Agentic Research**: Background research tasks and dependency graphs.
- **Immersive Reading**: Circle-to-ask on PDFs and precision context scope.
- **Smart Model Matching**: Automatically switch models based on capabilities (Images, Logic).
- **MCP Tools**: Connect Note Synapse to external tools (Google Drive, Slack, etc).

Productivity

- **Web Clipper**: Readability vs Raw HTML (for Vibe Coding).
- **Calendar & Tasks**: Drag-and-drop scheduling.

Developer

- **User Apps & Vibe Coding**: Build tools using `window.Synapse` .
- **Architecture**: Local-first design and HTTP/3.



Examples

- [Weather Dashboard](#)
- [SQL Stats](#)

Hands-On: Tag Algebra (Virtual Folders)

Organize your notes without ever moving a file.

The Concept: Filters Create Hierarchy

Tags in Synapse are **flat**. `#project` and `#marketing` are just separate labels. They do **not** use file paths like `#project/marketing`.

Instead, **Hierarchy is defined by Filter Algebra**: * **Filter A**: Notes with `#project`. * **Filter B**: Notes with `#project` AND `#marketing`. * **Result**: Synapse sees that Filter B is a *subset* of Filter A, so it automatically nests B under A in the visual tree.

Problem

You want to organize 50 notes about “AI Learning” into sub-topics like “LLMs” and “Vision”, but you don’t want to manage complex folder structures.

Walkthrough: Creating a “Virtual” Hierarchy

1. Tag Freely:

- Tag general notes with `#AI`.
- Tag specific notes with `#AI` and `#LLM`.
- Note: You do NOT need to name the tag `AI/LLM`. Just use two separate tags.

2. Create Parent Filter:

- Create a filter named “AI” that includes tag `#AI` .

3. Create Child Filter:

- Create a filter named “LLMs” that includes tags `#AI` and `#LLM` .

4. **Result:** In the Filter Strip, “LLMs” automatically tucks inside “AI” because it is a more specific subset.

Screenshot Placeholder: [Image of Filter Strip showing ‘LLMs’ nested under ‘AI’ because of tag containment.]

Tag Filters: Your Saved Views

While you can manually select tags, **Filters** let you save complex combinations (Algebra) for one-tap access.

1. The Filter Strip

Located at the top of your note list, the **Filter Strip** holds all your saved views. - **Add (+)**: Tap the plus icon to create a new filter. You can define it by: - **Tags**: Include or exclude specific tags. - **Text**: Must contain specific keywords.

2. Hierarchy Mode (Auto-Organization)

If you have many filters, the strip can get cluttered. Use **Hierarchy Mode** to organize them automatically. - **Toggle**: Tap the **Tree Icon** (`account_tree`) on the far left of the strip. - **Full Tree View: Long Press** the Tree Icon to open a popup dialog showing your entire filter hierarchy as a classic file-system tree. - **Logic**: Synapse detects “Algebraic Parents”. - If Filter A uses `#project` ... - And Filter B uses `#project` AND `#dev` ... - Then **Filter B becomes a child of Filter A**. - **Navigation**: Child filters are hidden to save space. Tap the **dropdown arrow** on the parent filter to reveal them.

3. Pinning for Speed

Keep your most-used filters accessible, even if they are deep in the hierarchy or hierarchy mode is active. 1. **Select:** Tap a filter to activate it. 2. **Pin:** Tap the **Pin Icon** (`push_pin`) inside the active filter chip. 3. **Result:** The filter moves to the **far left** of the strip and stays visible at all times, bypassing the hierarchy collapse.

Screenshot Placeholder: [Image of the Filter Strip showing a Pinned filter on the left, and a Parent filter with a dropdown arrow for hierarchy.]

Automation: Associated Tag Prompts

Tags can trigger AI work for you. You can associate a specific **Prompt** with a tag, so that when you apply it, Synapse automatically transforms the note.

Use Case: Research Papers

Imagine you save PDFs of research papers. You want to extract the same key info from every paper. 1. **Configure:** - Go to **Tag Management** (or long-press a tag in the filter list). - Tap the **Brain Icon** (`psychology`) next to the `#paper` tag. - Enter a Prompt: *“Summarize the key contributions, limitations, and related work of this paper.”* 2. **Apply:** - Open a new note containing a paper. - Add the tag `#paper` . 3. **Result:** Synapse detects the associated prompt and automatically runs it against the note content, appending the AI-generated summary to your note.

Hands-On: Web Clipper & API Vibe Coding

Stop copy-pasting. Start clipping.

The “Readability” Toggle

Problem

You want to read a long article in Note Synapse, but the “Raw” view is full of ads, popups, and broken layout scripts.

Walkthrough

1. **Open Browser:** Go to any article (e.g., a Hacker News link).
2. **Share:** Tap the system **Share** button -> **Note Synapse**.
3. **The Switch:**
 - **ON:** Synapse strips all junk. *Why?* Better for reading and standardizing the text for the AI.
 - **OFF:** Synapse keeps the raw HTML. *Why?* Better for “Vibe Coding” where you need the exact table structure or API code blocks.
4. **Save.**

Dealing with Images

Images are tricky. Here are the two ways Note Synapse handles them.

Method 1: The “Rich” Clip

If you use the Web Clipper with **Readability ON**, Synapse tries to download the main article images as **Attachments**. - They appear at the bottom of the note. - They act as local files (great for offline).

Method 2: Fetching Later

Sometimes the clipper misses images, or you clipped in “Raw” mode (where images are just URL links). 1. Open the Note. 2. Tap the **Examples/Menu** (`more_vert`) icon. 3. Select **“Fetch Remote Images”**. 4. **Result:** The app scans the note for `` links, downloads them, and converts them to local attachments

``.

Hands-On: Clipping API Docs for Vibe Coding

Usage

You want to build an app that uses the **Stripe API**, but the AI doesn't know the latest endpoints.

1. Navigate to the Stripe API reference for "Create Charge".
2. **Share** to Note Synapse.
3. **Turn Readability OFF**.
 - *Why?* You want the raw table structures and code blocks so the AI can understand the exact JSON schema.
4. **Use it:** Now use this note as context to say "Write me a Stripe dashboard".

Hands-On: Calendar & Time Blocking

Stop managing tasks in a list. Manage them in *time*.

Drag & Drop Rescheduling

Problem

You have 5 tasks due today and you know you can't do them all. Changing the due date on each one via a dropdown menu is slow.

Walkthrough

1. Open the **Calendar View** (Calendar icon).
2. You will see your tasks as blocks on the timeline.
3. **Long Press** on a task block.

4. **Drag** it to tomorrow, or next week.
5. **Drop**. The due date is instantly updated.

Screenshot Placeholder: [Image of the Calendar view, showing a task being dragged from 'Today' to 'Tomorrow'.]

The Timeline View (GTD)

1. Switch to **Timeline View** (List icon).
2. This shows a linear stream of *all* tasks across *all* notes.
3. **Filter**: Tap the Funnel icon to show only “High Priority” tasks.
4. **Check off**: Tap the circle to complete. It updates the original note instantly.

Tree Conversations: Git for Thoughts

Don't let a bad turn ruin a good conversation. Note Synapse treats every conversation as a **Tree**, allowing you to explore multiple possibilities without losing your place.

The “Forking” Workflow

Problem

You are debating architecture with the AI. It made a great point 5 messages ago, but then went down a rabbit hole about database drivers. You want to go back to that great point and explore a different direction.

Solution: Fork It

1. Scroll back to the **Message** where the AI made the good point (or where you want to change your reply).

2. Tap the **Fork Icon** (`call_split`) on the message bubble.
3. **Result:** A new “Branch” is created.
 - The old conversation is safe.
 - You are now in a fresh timeline starting from that exact moment.
 - The current thread is automatically highlighted in the Tree View.

Screenshot Placeholder: [Image of a chat message bubble with the Fork icon highlighted.]

Navigating the Tree

Tap the **Tree Icon** (`account_tree`) in the top-right of the chat screen to see the bigger picture.

The Graph View

- **Green Node:** Your current location.
- **Blue Nodes:** Other branches.
- **Edges:** Show the history flow.
- **Tap** any node to jump instantly to that version of reality.

Multi-Select Actions

You can manage multiple branches at once: 1. **Long Press** a node to enter selection mode. 2. Tap other nodes to add them to your selection. 3. Use the top-right menu for actions: * **Create Note** (`note_add`): Compile the selected conversation parts into a single permanent note. This is perfect for saving the “good parts” of a brainstorming session. * **New Conversation** (`add`): Start a brand new conversation using the selected nodes as context. You can even use an **AI Transformation** to summarize them first (e.g., “Summarize these 3 branches into a requirements doc”).

Use Case: Text Adventure Game

Imagine playing an RPG with the AI. 1. **Choice A**: You attack the dragon. -> *You die*. 2. **Rewind**: Go back to the message before the choice. 3. **Fork**: Choose “Choice B: Negotiate”. -> *You succeed*. 4. **Tree View**: You can now see both timelines side-by-side and effectively “save scum” your way to victory.

Context Pruning

Sometimes the AI generates massive output (e.g., a 500-line log analysis tool output) that clogs up your context window and wastes tokens.

1. Tap the **Tool Icon** (`build`) on the message with the large output.
2. Select “**Strip Tool Output**” (or “Exclude Message”).
3. **Result**: The message remains in the visual history, but its heavy content is removed from the AI’s “memory” for future turns. This keeps your conversation fast and focused.

Hands-On: Immersive Reading (PDFs)

Study complex papers without leaving your flow.

The “Circle to Ask” Gesture

Problem

You are reading a PDF about Quantum Mechanics. You see a formula `$H(t) \mid \psi(t) \rangle$` . You have no idea what `$H(t)$` specifically means in *this* context.

Entering Immersive Mode

1. **Open any Note.**
2. Tap the **Book Icon** (`chrome_reader_mode`) in the top toolbar.
3. **Result:** You enter a unified viewing experience where the distractions of the editor are removed.

Working with PDFs

Immersive Mode is designed for deep reading of attached PDFs.

1. Opening a PDF

This is the tricky part. By default, Immersive Mode shows the note text. 1. Tap the **Outline Icon** (`format_list_bulleted`) in the top-right. 2. You will see a list of sections. 3. **Select the PDF:** If your note has a PDF attachment, it appears here. Tap it to switch the view to the PDF.

2. The Side Handle (AI Panel)

You will see a small “Handle” floating on the side or bottom of the screen. - **Tap** to expand the AI Chat panel. - **Drag** to resize the panel (give yourself more reading space vs. chat space). - **Context:** The AI in this panel automatically sees what you are reading.

The “Circle to Ask” Gesture

Problem

You are reading a critical formula `H(t) | ψ(t)>` and need an explanation.

Walkthrough

1. In Immersive Mode (with PDF open), select the **Pen Tool** (or just use your Apple Pencil).
2. **Draw a Red Circle** around the formula.
3. Write in the prompt: “Explain this parameter”.
4. **Result:** The AI analyzes the *pixels* inside your circle to understand symbols that normal text extraction misses.

The Scratchpad

Problem

You are reading and have a flash of insight, or need to jot down key terms, but you don’t want to leave the reading view or start a full chat session.

Solution

The Scratchpad is a temporary buffer that sits “between” your reading and your permanent note.

1. **Toggle:** Tap the **Note Icon** inside the AI Panel (near the input bar) to switch from “Chat Mode” to “Scratchpad Mode”.
 - *Indicator:* The input bar hint changes to “Send to scratchpad”.
2. **Capture:** Type notes, copy-paste snippets from the PDF, or even scribble with the pen.
3. **Use It:**
 - **Context for AI:** Check the box “**Include scratchpad in chat context**”. Now, when you switch back to Chat, the AI knows everything you just jotted down.
 - **Save to Note:** Tap “**Add to Note**” to append your scratchpad items permanently to the underlying note.

Screenshot Placeholder: [Image of the AI Panel in ‘Scratchpad Mode’, showing a list of bullet points and the ‘Include in Chat’ checkbox.]

Hands-On: Limiting Context Scope

Problem

You are asking the AI to summarize Chapter 5, but it keeps hallucinating facts from Chapter 1.

Walkthrough

1. In the Immersive Chat, tap the **Context Settings** icon (Gear/Page symbol).
2. **Scope**: Switch from “Whole Document” to “**Current Chapter**”.
3. **Range**: Set the page slider to “**+/- 0 pages**”.
4. **Result**: The AI is now blind to everything except exactly what you are looking at. Hallucinations drop to near zero.

Connecting to External Tools (MCP)

Note Synapse uses the **Model Context Protocol (MCP)** to talk to the outside world.

What is MCP?

MCP (Model Context Protocol) is an open standard that allows AI assistants to connect to external data and tools.

Supported Transports

1. **SSE (Server-Sent Events)**: Best for remote servers.
2. **StreamHTTP**: Note Synapse supports MCP over HTTPStreaming

How to Connect: The Definitive Guide

1. The “Magic” Way (Auto-Discovery)

If your MCP server supports OpenID Connect (OIDC) discovery, Note Synapse can configure itself.

1. **Add Endpoint:** Settings -> MCP -> Add (+).
2. **Transport:** Select `SSE` (or HTTP).
3. **Auth (OAuth Tab):**
 - Click the **Magic Wand Icon** (`auto_fix_high`).
 - Enter the base URL of your server (e.g., `https://my-mcp-server.com`).
 - Tap **Discover**.
4. **Result:** Synapse fetches the meta-data and auto-fills `Authorization URL`, `Token URL`, and `Scopes`.
5. **Login:** Click the **Login** icon button to authenticate and capture your first token.
6. **Save:** Click Create/Save.

Screenshot Placeholder: [Image of the OAuth Discovery dialog showing the “Auto Configure” button and auto-filled fields.]

2. The “Simple” Way (Token Auth)

For servers that just need a static secret.

1. **Add Endpoint:** Settings -> MCP -> Add (+).
2. **Auth (Token Tab):**
 - Paste your Bearer token (e.g., `sk-proj-...`).
3. **Save.**

Screenshot Placeholder: [Image of the MCP Settings dialog with the ‘Token’ tab selected and a token pasted in.]

3. The “Manual” Way (Advanced OAuth)

If auto-discovery fails, you can enter the OAuth details manually.

1. **Auth (OAuth Tab):** Fill in the fields manually.

- **Auth URL:**

- `https://accounts.google.com/o/oauth2/v2/auth`

- **Token URL:** `https://oauth2.googleapis.com/token`

- **Client ID:** Your registered app ID.

- **Scopes:** Space-separated list (e.g.,

- `openid profile email`).

- **PKCE:** Check this if your server handles Proof Key for Code Exchange (standard for mobile apps).

2. **Login:** You **MUST** click the Login button *inside* this dialog to verify the flow before saving.

Debugging Tools (AI Introspection)

If a tool isn't working, use the **AI Debug Overlay**: 1. Enable it in **Settings -> AI Debug Overlay**. 2. Look for "MCP: Call Tool" requests. 3. Inspect the `Result` payload to see raw JSON-RPC errors.

Screenshot Placeholder: [Image of the AI Debug Overlay showing a successful MCP tool call and its JSON response.]

Smart Model Matching (Multi-Capability Selection)

Note Synapse solves the "Model Dilemma": *Do I use the cheap, fast model (e.g., Gemini Flash) or the smart, expensive one (e.g., GPT-4o)?*

With **Smart Matching**, you don't have to choose. You set a default (cheap) model, and Synapse automatically swaps in a powerful model **only when required**.

Problem

You default to a text-only or lightweight model (e.g., local Llama on your Mac, or Gemini Flash) for speed and cost. But occasionally, you drag in an **Image** or a **PDF**, or ask for **Code Generation**. Your default model fails or gives poor results.

Solution: Capability-Based Priority

You define a “Priority List” of models. When you send a message, Synapse analyzes it: *“This message contains an Image and requires Code Generation.”*

It then scans your models in order: 1. **Active Model** (Default): Does it support Images + Code? -> No. 2. **Priority 1** (e.g., Claude 3.5 Sonnet): Does it support Images + Code? -> **Yes**. 3. **Result**: Synapse automatically uses Priority 1 for *this specific turn*.

The next message (text only) reverts to your lightweight default.

How to Configure

1. Go to **Settings -> AI Settings -> Model Preference**.
2. **Defaults**: Set your “Active Model” (main chat screen) to your preferred daily driver (e.g., Flash).
3. **Priority List**: Drag and drop powerful models from “Available” to “Priority List”.
 - *Recommendation*: Put your most capable “Swiss Army Knife” model at the top (e.g., GPT-4o or Claude 3.5 Sonnet).
 - *Recommendation*: Put specialized models below (e.g., a dedicated Image Generator).

Screenshot Placeholder: [Image of Model Preference screen showing a ‘Priority List’ with ‘Gemini Pro’ above ‘DALL-E 3’, and ‘Llama Local’ as the active default.]

Matching Logic

The selector follows this strict order:

1. **Perfect Match:** Scans the Priority List for the first model that supports **ALL** required capabilities (Images, Audio, Docs, etc.) found in your message.
2. **Critical Media Fallback:** If no perfect match is found but your message has Media (Images/Video), it prioritizes *any* model that can see the media, even if it lacks other skills to avoid “I can’t see that” errors.
3. **Fallback:** If nothing matches, it uses your Active Default.

Supported Capabilities

-  **Images:** Analyzing photos/screenshots.
-  **Documents:** Reading PDFs or text files.
-  **Video:** Analyzing video files.
-  **Audio:** Listening to voice notes.
-  **Image Gen:** Creating images (DALL-E / Imagen).
-  **Code Gen:** Writing complex code.

Hands-On: Agentic Research

Turn Note Synapse into a background worker.

The “Research Agent” Workflow

Problem

You want to know “How does Note Synapse handle dependency injection?” but you don’t want to search through 50 files yourself.

Walkthrough

1. **Start Agent Mode:** Toggle the toggle switch from “Chat” to “Agent”.
2. **Define Objective:** Type: *“Analyze the codebase and explain the Service Locator pattern usage.”*
3. **Attach Context:**
 - Tap the **Paperclip**.
 - Add `lib/services/service_locator.dart`.
 - *Tip:* Giving the agent a starting point helps it run faster.
4. **Run:** Tap Send.
5. **Multi-Tasking:**
 - **Android Users:** You can minimize the app. A “Foreground Service” notification will appear (“Synapse Agent is Thinking...”).
 - Go check your email. The agent will continue reading files and reasoning in the background.

Reviewing the Plan

1. Before executing, the Agent will propose a **Plan** (e.g., “1. List files, 2. Read File A, 3. Read File B”).
2. **Edit the Plan:** You can delete steps that seem irrelevant (e.g., “Don’t read `main.dart`, I know that’s not it”).
3. **Approve:** Tap “Execute” to start the background work.

Screenshot Placeholder: [Image of the Agent Plan Review screen, showing a list of proposed tasks with generic checkboxes.]

Hands-On: Block-Based Editing

Note Synapse treats every paragraph, header, and list item as a separate “block”. This gives you power features usually found in tools like Notion.

The “Hidden” Drag Gesture

Problem

You want to reorder a paragraph or convert it to a different type, but you don't see any handles.

Solution

The “Edit” icon (Pen) in the top-right is actually a **Draggable Tool**.

Walkthrough

1. Open any note in **Viewing Mode** (not editing text).
2. **Long Press** the  (Pen) icon in the top toolbar.
3. **Drag** it onto any block of text in your note.
4. **Drop** it.
5. A menu will appear allowing you to:
 - **Edit** just that block.
 - **Delete** the block.
 - **Reorder** (if multiple blocks selected).
 - **Convert** (e.g., Text -> Checkbox).

Screenshot Placeholder: [Image showing the Pen icon being dragged from the app bar, with a ghost icon floating over a paragraph, highlighting the drop target.]

Reordering Blocks

1. Once you select a block (via the Pen Drop method), you enter **Block Selection Mode**.
2. Tap other blocks to add them to the selection.
3. Use the **Up/Down arrows** in the floating menu to move the entire selection.

Architecture

Technical Stack

- **Framework:** Flutter (Dart)
- **Database:** SQLite (via `sqflite` FFI)
- **AI:** Gemini API / OpenAI API (Standardized Interface)
- **Network:** HTTP/1.1 + HTTP/3 (QUIC)

Network Resilience (HTTP/3)

LLM inference requests are long-lived and fragile. Simple timeouts break agents. Note Synapse uses a custom networking stack including **HTTP/3 (QUIC)** to ensure stability on flaky mobile networks (e.g., switching from WiFi to 5G while an agent is thinking). -

Libraries: Custom Rust-based HTTP client bridged via FFI.

Agent Architecture

Agents are not just “loops”. - **Android:** Agents run as **Foreground Services**. This means you can start a deep research task, switch to another app, lock your phone, and the agent continues to work. -

State Machine: Notes are the state. An agent reads a note, performs an action, and updates the note. This provides “Time Travel” debugging for agents.

Local-First Data Model

Your data is yours. - **DB Path:** `note_synapse.db` in your App

Documents folder. - **Attachments:** Stored as raw files in

`attachments/`. - **Querying:** You can open the DB with any SQLite viewer. The schema is straightforward: - `notes` : id, title, content

(markdown), type - `note_tags` : many-to-many link -
`relationships` : graph edges (parent/child, related)

User Apps & “Vibe Coding”

Turn your notes into software. User Apps are HTML/JS applications that run *inside* Note Synapse with privileged access to your data and AI.

The “Vibe Coding” Workflow

You don’t need to write code to build a tool. You just need the *docs*.

Tutorial: “Build a Weather Dashboard”

1. **Clip:** Use the Web Clipper to save the `OpenWeatherMap API` documentation page.
2. **Context:** Open a new Chat. Add the “OpenWeatherMap API” note as context.
3. **Prompt:** > “Create a User App called ‘Weather Dash’. It should use `window.Synapse.proxyFetch` to call the API. Ask the user for a city, then display the current temp and icon. Use the API key provided in the system prompt.”
4. **Result:** Note Synapse generates a fully functional HTML/JS app. Click “Run” to test it instantly.

Screenshot Placeholder: [Image showing the ‘Create User App’ dialog with the API documentation attached as context.]

`window.Synapse`

API Reference

1. Database Access (SQL)

Directly query your `note_synapse.db` SQLite database.

```
// Get tasks due today
const today = new Date().toISOString().split('T')[0];
const tasks = await window.Synapse.runQuery(
  `SELECT * FROM notes
   WHERE type = 'task'
   AND json_extract(content, '$.dueDate') LIKE
     '${today}%`;
);
```

2. State Management

Apps can persist state between sessions.

```
// Save preferences
await window.Synapse.storeAppState({ theme: 'dark',
  lastCity: 'London' });

// Load preferences
const state = await window.Synapse.loadAppState();
console.log(state.lastCity); // 'London'
```

3. AI Integration (Multi-Part)

```
const response = await window.Synapse.chatAI(
  "Analyze this data...",
  {
    temperature: 0.7,
    model_hint: ['gemini-pro']
  }
);
```

3. Network Proxy

Bypass CORS (Cross-Origin Resource Sharing) restrictions.

```
// Fetch data from any API, even if it doesn't support CORS  
  
const data = await window.Synapse.proxyFetch(  
  "https://api.example.com/data"  
);
```

4. Note Manipulation

```
// Save a new note  
await window.Synapse.saveNotes([  
  { title: "Report", content: "..."}  
]);  
  
// Open a note in the editor  
await window.Synapse.openNote("note-id-123");
```

Security Model

- **Read-Only by Default:** Apps can read DB/Notes by default.
- **Modification Approval:** The first time an app tries to `deleteNotes` or run a `Write` SQL query, the user is prompted to approve the session.